

NEXUSTRADE

AI-Powered Multi-Model Stock Trading Simulator

Project Documentation

Generated October 08, 2026

Stack	Next.js 16 · React 19 · FastAPI · Python · SQLite
AI Models	Claude Sonnet · GPT-4o · Gemini 1.5 Pro · DeepSeek Chat
Market Data	Finnhub (real-time) with deterministic mock fallback
3D / Visuals	Three.js particle field · lightweight-charts v5
Paper Money	\$100,000 virtual USD starting balance
Repository	github.com/sohangowda11/TradeWithAi

1. Introduction to Stock Trading

What Is a Stock?

A stock (also called a share or equity) represents a fractional ownership stake in a company. When a company wants to raise capital, it can divide itself into millions of equal pieces and sell them to the public through a process called an IPO (Initial Public Offering). Each piece is one share. If you own 100 shares of a company that has 1,000,000 shares outstanding, you own 0.01% of that company.

How Prices Move

Stock prices are determined by supply and demand on an exchange. When more people want to buy a stock than sell it, the price rises. When more people want to sell, it falls. The underlying drivers of this supply/demand are: company earnings reports, macroeconomic data (interest rates, inflation, GDP), news and sentiment, analyst upgrades/downgrades, sector rotation, and broader market trends.

Key Terms

Term	Definition
Market Cap	Total value of all shares = Share Price × Shares Outstanding
Volatility	How much a stock's price swings. High volatility = larger moves, more risk
Bid / Ask	Bid = highest price a buyer will pay. Ask = lowest a seller will accept
Volume	Number of shares traded in a period. High volume confirms price moves
P/E Ratio	Price-to-Earnings: how much investors pay per \$1 of earnings
Bull / Bear	Bull market = prices rising broadly. Bear market = prices falling broadly
Liquidity	How easily you can buy/sell without moving the price. Large caps = high liquidity

Types of Risk

Market risk: the whole market falls (e.g., 2008 financial crisis). Sector risk: a specific industry is hurt (e.g., oil stocks when crude prices crash). Company-specific risk: a single company has bad news (earnings miss, lawsuit, CEO resignation). Liquidity risk: you cannot sell when you want to. The key insight behind diversification is that company-specific and sector risks can be reduced by holding many different stocks.

Why Paper Trading Matters

Paper trading means simulating trades with fake money but using real market prices. It lets you develop intuition, test strategies, and learn from mistakes without losing real capital. Professional traders often paper-trade a new strategy for months before committing real money. NexusTrade is a paper trading simulator — all money is virtual, all trades are simulated, and no real brokerage integration exists.

2. The AI Voting & Discussion System

Overview

The core innovation of NexusTrade is treating stock analysis as a multi-agent deliberation problem. Rather than relying on a single AI model's opinion, the system queries four different large language models in parallel, collects their independent analyses, then runs a second 'discussion' round where each model sees what the others said and can revise its view. The final signal is an aggregate of these revised votes.

What Each Model Receives (The Prompt)

Every model gets an identical structured prompt containing:

- Ticker symbol being analyzed
- Current price, day change %, high, and low
- RSI (14-period), MACD trend, SMA 20, SMA 50, volume trend, 5-day momentum
- Up to 5 recent news headlines from Finnhub (last 48 hours)
- Explicit JSON output format requirement

What Each Model Returns

Models are instructed to respond with strict JSON only:

```
{"direction": "up" or "down", "confidence": 0-100, "risk_level": "low"/"medium"/"high",  
"reasoning": "2-3 sentence explanation"}
```

Round 1: Independent Analysis

All four models are called simultaneously using Python's `asyncio.gather()`, meaning they run in true parallel (limited only by network latency). Each model analyzes the same data independently, with no knowledge of what the others are saying. This independence is critical — it avoids 'groupthink' where one influential opinion anchors all the others.

Round 2: The Discussion

After Round 1, each model is sent a new prompt that includes its own Round 1 vote plus a summary of the other three models' directions, confidence levels, and reasoning. The model is asked: 'Considering their perspectives, would you revise your prediction?' This simulates how a real investment committee might work — analysts present their views, hear challenges, and either hold their ground or update based on new arguments. Round 2 votes are what the final signal is based on.

Why Only 2 Rounds?

Each round costs real money in API tokens. With 4 models and 2 rounds, that is 8 API calls per analysis. Research on multi-agent deliberation shows that most information exchange happens in the first discussion round — additional rounds produce diminishing returns while costs compound. The system is designed to cap at 2 rounds to balance insight quality against cost.

Aggregation Algorithm

Signal	How Computed
Final Direction	Majority vote among Round 2 directions. Tie = 'uncertain'
Final Confidence	Weighted average of confidence scores from the majority side
Final Risk	Worst-case escalation: if any model says 'high', result is 'high'
Consensus Score	% of models that voted with the majority. 100% = full agreement

Mock Mode

If an API key is missing or a model call fails, the system falls back to a deterministic mock response. The mock is seeded by the model name + ticker symbol, so it produces the same 'fake vote' every time for that combination within a session. This means the UI is fully functional with zero API keys — great for development and demonstration.

3. Paper Trading Engine

Virtual Wallet

Every user starts with \$100,000 in virtual USD. This balance is stored in a SQLite database (the Portfolio table) and is persistent across sessions. You can manually add or withdraw funds at any time through the portfolio panel in the UI — there is no upper limit, because it is fake money.

Executing a Buy

When you buy shares: (1) the system checks you have enough cash, (2) deducts the total cost (shares × price) from your cash balance, (3) creates or updates a Holding record for that ticker. If you already own shares, the new purchase is averaged into your existing position using a weighted average cost basis formula:

$$\text{new_avg_price} = (\text{old_avg} \times \text{old_shares} + \text{new_price} \times \text{new_shares}) / (\text{old_shares} + \text{new_shares})$$

Executing a Sell

When you sell shares: (1) the system checks you actually hold enough shares to sell, (2) calculates realized P&L; using the FIFO average cost basis: $\text{P\&L} = (\text{current_price} - \text{avg_cost}) \times \text{shares_sold}$, (3) adds the proceeds back to your cash, (4) reduces the holding. If shares reach zero, the holding record is deleted.

Position Sizing: The Kelly Criterion

The AI system can recommend how many shares to buy, not just whether to buy. It uses a simplified Kelly Criterion — a mathematical formula from gambling theory that tells you the optimal fraction of your bankroll to bet given your edge:

$$\text{Kelly fraction } f = \max(0, 2p - 1)$$

Where p = confidence / 100. So 70% confidence → $f = 0.40$ (bet 40% of available cash). This raw fraction is then capped by a risk tier multiplier to prevent catastrophic overexposure:

Risk Level	Cap	Example: 70% confidence, \$50,000 cash
Low	15%	$\min(40\%, 15\%) \times \$50,000 = \$7,500$ position
Medium	10%	$\min(40\%, 10\%) \times \$50,000 = \$5,000$ position
High	5%	$\min(40\%, 5\%) \times \$50,000 = \$2,500$ position

P&L; Tracking

Unrealized P&L;: $(\text{current_price} - \text{avg_cost}) \times \text{shares held}$. Updates every time the watchlist quote refreshes (every 30 seconds). Realized P&L;: calculated at sell time and stored permanently in the Trade record. All-time P&L;: $\text{total_portfolio_value} - \$100,000$ starting balance. Win rate: percentage of completed sell trades where realized P&L; was positive.

4. System Architecture

Frontend

Technology	Purpose
Next.js 16	React framework, App Router, Server + Client components
React 19	UI rendering, concurrent features
TypeScript	Type safety across all components and API calls
Tailwind CSS v4	Utility classes with @theme CSS-first configuration
Zustand	Persistent client state: wallet, holdings, trades, watchlist
React Query	Server state: API data fetching, caching, background refresh
Three.js	3D particle field in the hero section
lightweight-charts v5	Professional candlestick charts (TradingView library)
framer-motion	Animated card entrances, loading states
Lucide React	Icon library
axios	HTTP client for backend API calls

Backend

Technology	Purpose
FastAPI	Async Python web framework, automatic OpenAPI docs
SQLAlchemy 2	ORM for database models and queries
SQLite	Local database (upgradeable to PostgreSQL for production)
Pydantic v2	Request/response validation and serialization
httpx	Async HTTP client for Finnhub API calls
APScheduler	Weekly model retraining cron job
scikit-learn	GradientBoostingClassifier for the meta-model
numpy	Technical indicator computations

Database Schema

Table	Key Columns
Portfolio	id, cash_balance, created_at, updated_at

Holding	id, portfolio_id, ticker, shares, avg_price
Trade	id, portfolio_id, ticker, action, shares, price, total, pnl, timestamp
AnalysisSession	id, ticker, final_direction, final_confidence, final_risk, consensus_score, price_at_analysis, price_24h
AIVote	id, session_id, model_name, direction, confidence, risk_level, reasoning, discussion_round, was_correct
ModelPerformance	id, model_name, ticker, correct_predictions, total_predictions, accuracy

Data Flow: Analyze a Stock

Step	Action
1	User clicks ANALYZE AAPL in the AI Voting Panel
2	Frontend calls POST /api/analysis/run {ticker: 'AAPL'}
3	Backend fetches quote, 60-day candles, and news from Finnhub in parallel
4	TechnicalAnalysisService computes RSI, MACD, Bollinger Bands, SMA, volume trend
5	ai_service builds the analysis prompt and calls all 4 models in parallel (Round 1)
6	Each model's vote is collected; models that fail fall back to mock responses
7	Discussion prompts built; all 4 models called again in parallel (Round 2)
8	Votes aggregated: majority direction, weighted confidence, risk escalation
9	AnalysisSession + 8 AIVote rows written to SQLite
10	Result JSON returned to frontend; Zustand store updated; UI animates in

5. Technical Indicators

RSI — Relative Strength Index

RSI measures how overbought or oversold a stock is on a 0-100 scale, using the last 14 trading days. Formula: $RSI = 100 - 100/(1 + RS)$, where $RS = \text{average gain over 14 days} / \text{average loss over 14 days}$. $RSI > 70$: stock may be overbought (price could pull back). $RSI < 30$: stock may be oversold (possible bounce). RSI around 50: neutral momentum.

MACD — Moving Average Convergence Divergence

MACD uses three exponential moving averages: the 12-day EMA, 26-day EMA, and a 9-day signal line. $MACD \text{ line} = EMA(12) - EMA(26)$. $\text{Signal line} = EMA(9) \text{ of the MACD line}$. $\text{Histogram} = MACD - \text{Signal}$. When the histogram is positive (MACD above signal), trend is bullish. When negative, trend is bearish. The MACD 'crossover' (when MACD crosses above signal) is a classic buy signal; crossing below is a sell signal.

Bollinger Bands

Bollinger Bands place an upper and lower envelope around the 20-day SMA, each 2 standard deviations away. $\text{Upper band} = SMA(20) + 2\sigma$. $\text{Lower band} = SMA(20) - 2\sigma$. When price touches or exceeds the upper band, the stock may be overbought. When it touches the lower band, it may be oversold. 'Band squeeze' (bands narrowing) signals that a large move is coming, though not which direction.

SMA — Simple Moving Average

$SMA(20)$ and $SMA(50)$ are the average closing prices over the last 20 and 50 days. Golden Cross: $SMA(50)$ crosses above $SMA(200)$ — historically bullish. Death Cross: $SMA(50)$ crosses below $SMA(200)$ — historically bearish. Price trading above $SMA(20)$ is generally considered short-term bullish.

Volume Trend

Volume trend compares the 5-day average volume against the 20-day average. Above average (ratio > 1.2): unusual interest, move is more likely to sustain. Below average (ratio < 0.8): low conviction, price move may reverse. Volume always confirms or denies a price move — a price rise on low volume is weak.

6. Custom Model Training

What 'Training Your Own Model' Actually Means

This is NOT training a new large language model from scratch — that would require millions of dollars and months of GPU time. Instead, NexusTrade trains a lightweight 'meta-model': a classifier that learns which AI models (or combinations of them) tend to be right. Think of it as a referee that has watched thousands of rounds and learned whose opinions to trust most.

The Feature Matrix

Each row in the training dataset represents one completed AnalysisSession where the 24-hour price outcome is already known. The features for each row are:

Feature	Description
claude_dir / gpt_dir / gemini_dir / deepseek_dir	Direction, 0=down, -1=no vote
claude_conf / gpt_conf / gemini_conf / deepseek_conf	Confidence / 100 (normalized 0-1)
claude_risk / gpt_risk / gemini_risk / deepseek_risk	Risk level, 1=medium, 2=high
consensus_score	Fraction of models that agreed with majority
weekday	Day of week (0=Mon to 6=Sun), normalized
hour	Hour of day (0-23), normalized — markets behave differently at open vs close

The Label

Label = 1 if the consensus direction prediction turned out to be correct 24 hours later (i.e., final_direction == 'up' AND price_24h_later > price_at_analysis, or vice versa). Label = 0 if the prediction was wrong.

Why GradientBoosting?

GradientBoostingClassifier from scikit-learn is ideal here because: the dataset is small (tabular rows, not images or text), it handles mixed feature types well, it is interpretable via feature importances (you can see which AI model's votes matter most), and it does not require a GPU. The model uses 100 estimators and max_depth=3 to prevent overfitting on small datasets.

Data Requirements

The system requires a minimum of 20 labeled samples (analysis sessions with known 24h outcomes) before training is attempted. With fewer samples, the model would just memorize the training data. As you use NexusTrade over days and weeks, the dataset grows. The system retrains automatically every Sunday at 2am via APScheduler.

Feature Importances

7. Setup & Running Locally

Prerequisites

- Node.js 18+ (for the Next.js frontend)
- Python 3.11+ (for the FastAPI backend)
- git (for version control)

Step-by-Step Setup

Step	Command / Action
1. Clone repo	git clone https://github.com/sohangowda11/TradeWithAi.git
2. Backend deps	cd TradeWithAi/backend → pip install -r requirements.txt
3. Create .env	copy backend\.env.example backend\.env (then fill in keys)
4. Start backend	cd backend → uvicorn app.main:app --reload --port 8000
5. Frontend deps	cd frontend → npm install
6. Start frontend	cd frontend → npm run dev
7. Open browser	http://localhost:3000

API Keys (all optional — mock mode works without them)

Service	Where to get it	Env var
Finnhub	finnhub.io → Dashboard → API Key	FINNHUB_API_KEY
Anthropic	console.anthropic.com → API Keys	ANTHROPIC_API_KEY
OpenAI	platform.openai.com → API Keys	OPENAI_API_KEY
Google	aistudio.google.com → Get API Key	GOOGLE_API_KEY
DeepSeek	platform.deepseek.com → API Keys	DEEPSEEK_API_KEY

8. Limitations & Disclaimers

NOT Financial Advice

NexusTrade is an educational simulation tool. Nothing in this application constitutes financial advice, investment recommendations, or trading signals for real markets. Do not make real investment decisions based on this system's output.

LLMs Do Not Have Real-Time Data

Claude, GPT-4o, Gemini, and DeepSeek were trained on historical text data with knowledge cutoffs. They reason from patterns in their training data, not from actual current market knowledge. Their 'analysis' is a synthesis of general financial reasoning, not privileged market intelligence.

Past Accuracy Does Not Predict Future Accuracy

Even if the custom meta-model shows high accuracy on historical votes, this does not guarantee future performance. Markets are non-stationary — conditions change, correlations break down, and black swan events invalidate all technical signals simultaneously.

Paper vs. Real Trading

Paper trading eliminates psychological factors (fear, greed, FOMO) that heavily influence real trading performance. A strategy that works perfectly in simulation often performs worse when real money is at stake.

API Rate Limits and Costs

Running 8 AI API calls per analysis across 4 providers can be expensive at scale. Each analysis session costs roughly \$0.02-0.50 in API tokens depending on providers used. The backend enforces a MAX_AI_COST_PER_ANALYSIS cap (\$0.50 by default) configurable in settings.

24-Hour Outcome Window

The system evaluates prediction accuracy by checking if the price moved in the predicted direction 24 hours after the analysis. This is a significant oversimplification — real trading involves multiple timeframes, stop losses, position sizing, and exit strategies.

SQLite for Development Only

The default database is SQLite. For any production deployment handling concurrent users, this should be upgraded to PostgreSQL (Supabase, Railway, or Neon all offer free tiers).